Intermediate
Azure Lab05
Terraform Checks and Validations

Contents

Overview

This lab introduces Terraform validation mechanisms (preconditions and postconditions)

Before You Begin

1. Ensure you have Owner or User Access Administrator permissions at the **subscription** level.
2. Azure CLI and Terraform must be installed.

3. Navigate to the lab directory:

   cd c:\azure-tf-int\labs\05

4. Update **main.tf** with your subscription id.

5. Update **terraform.tfvars,** adding a unique suffix to the storage account name.

Phase 1 – VM Size Precondition Check

Purpose

What we are doing:
Using Terraform's validation block inside a variable definition to restrict input values for VM sizes.

Why it matters:
In cloud environments with cost controls or policy restrictions, not all VM sizes are allowed. This validation protects the user from accidentally requesting expensive or disallowed SKUs — acting as a **client-side policy enforcement** before Terraform even contacts Azure.

Steps

1. Review the provided variables.tf file. Two variables are declared, vm_size and storage_account_name. Both have validations, the vm size must be one of the 4 listed and the storage account name must begin with “lab”

6. Review the provided terraform.tfvars file. Values for both variables are present. The vm size listed is one of the four permitted sizes and the storage account name begins with ‘lab’.

7. In terraform.tfvars; change the vm size from **"Standard_B2s”**, to **"Standard_B4s”**

8. Run **terraform init** and **terraform plan**. This should **fail** with an error indicating an invalid machine size choice...

```
Error: Invalid value for variable

  on terraform.tfvars line 1:
   1: vm_size = "Standard_B4S"

      var.vm_size is "Standard_B4S"

Only the following VM sizes are allowed in this lab: Standard_D2s_V3, Standard_DS2_V2, Standard_B2S, Standard_DS3_v2.

This was checked by the validation rule at variables.tf:4,3-13.
```

9. In terraform.tfvars; reset the vm size back to **Standard_B2s**

Phase 2 – Storage Account Name Validation

Purpose

What we are doing:

1. **Precondition** ensures the **input value** for the storage account name starts with "lab".
2. **Postcondition** ensures that after Terraform's logic runs, the **actual deployed name** still meets this rule.

Why it matters:
Sometimes, logic inside a Terraform configuration (e.g. using replace() or lower()) transforms a valid input into an invalid output.
The precondition validates the **user's input**, while the postcondition verifies the **final evaluated result**. This double-check ensures that even advanced logic does not break compliance rules unintentionally.

Steps

1. In **terraform.tfvars;** remove '**lab**' from the storage account name
2. Run **terraform plan.** This should fail pre-condition checks with an error indicating an invalid name...

```
Error: Invalid value for variable

  on terraform.tfvars line 2:
   2: storage_account_name = "storage3456123"

      var.storage_account_name is "storage3456123"

Storage account name must start with 'lab'.
```

3. Revert the storage account name so it begins with ‘lab’ and thus complies with the validation check. Planning should now succeed as before.

4. You will now introduce an error whereby the storage account name is altered so that, whilst the initial value passes validation check, the eventual value, altered by some logic processes, is not valid.

4. In main.tf, navigate to line 66 and change the **second** instance of ‘**lab**’ to ‘**test**’. This regenerates a new storage account name that varies from the name derived from the variable.

5. Run **terraform plan.** This should fail post-condition checks with an error indicating an invalid name...

```
Error: Resource postcondition failed

  on main.tf line 83, in resource "azurerm_storage_account" "storage":
  83:       condition     = can(regex("^lab", self.name))

      self.name is "teststorage3456123"

Evaluated storage name does not start with 'lab'.
```

6. Undo the changes to main.tf line 66

7. Run **terraform plan** followed by **terraform apply** to verify that the valid resources deploy successfully

Phase 3 – Cleanup

1. Run **terraform destroy**

Congratulations, you have completed this lab

Copyright